



Real-Time Operating System

Jong-Chan Kim

Graduate School of Automotive Engineering



국민대학교
KOOKMIN UNIVERSITY

Erika Enterprise



- 이탈리아 EVIDENCE에서 개발된 오픈소스 OSEK/VDX RTOS
- 듀얼 라이선스 정책 (오픈소스 라이선스 + 상용 라이선스)
- RTOS 연구 및 교육에 널리 사용
- GitHub 리포



evidence / erika3 Public

Watch 25 Fork 72 Star 155

Code Issues 2 Pull requests 1 Actions Projects Wiki Security Insights

master 1 branch 15 tags Go to file Add file Code

eguidieri [tricore] Fixed interrupt regression in... 85c445d on 13 Sep 2019 245 commits

contrib	[kernel] PostTaskHook called twice fixed and Initial ...	3 years ago
doc	[kernel] Fixed WaitEvent, in case of multicore enviro...	4 years ago
mk	[kernel] Fixed WaitEvent, in case of multicore enviro...	4 years ago
pkg	[tricore] Fixed interrupt regression introduced by co...	3 years ago
LICENSE.TXT	Update LICENSE to all files	4 years ago
README.md	[kernel] Fixed WaitEvent, in case of multicore enviro...	4 years ago
THIRDPARTY.TXT	[THIRDPARTY] Rephrased and fixed the statement r...	3 years ago

About

ERIKA Enterprise v3 RTOS

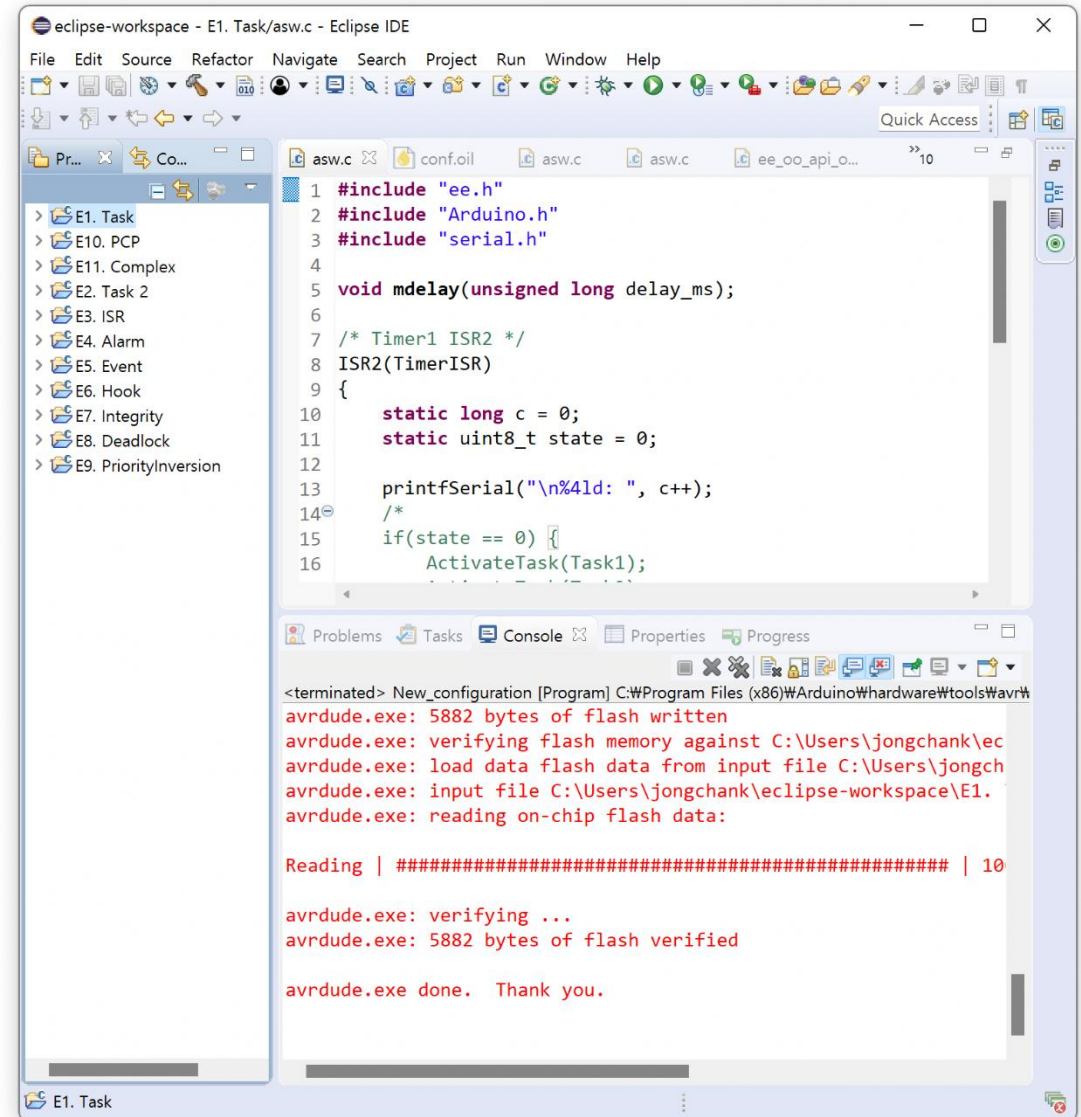
www.erika-enterprise.com

arduino avr arm real-time
cortex-m x86-64 xen rtos
cortex-a autosar osek dspic
aurix tricore-development cortex-r5f
erika-enterprise jailhouse

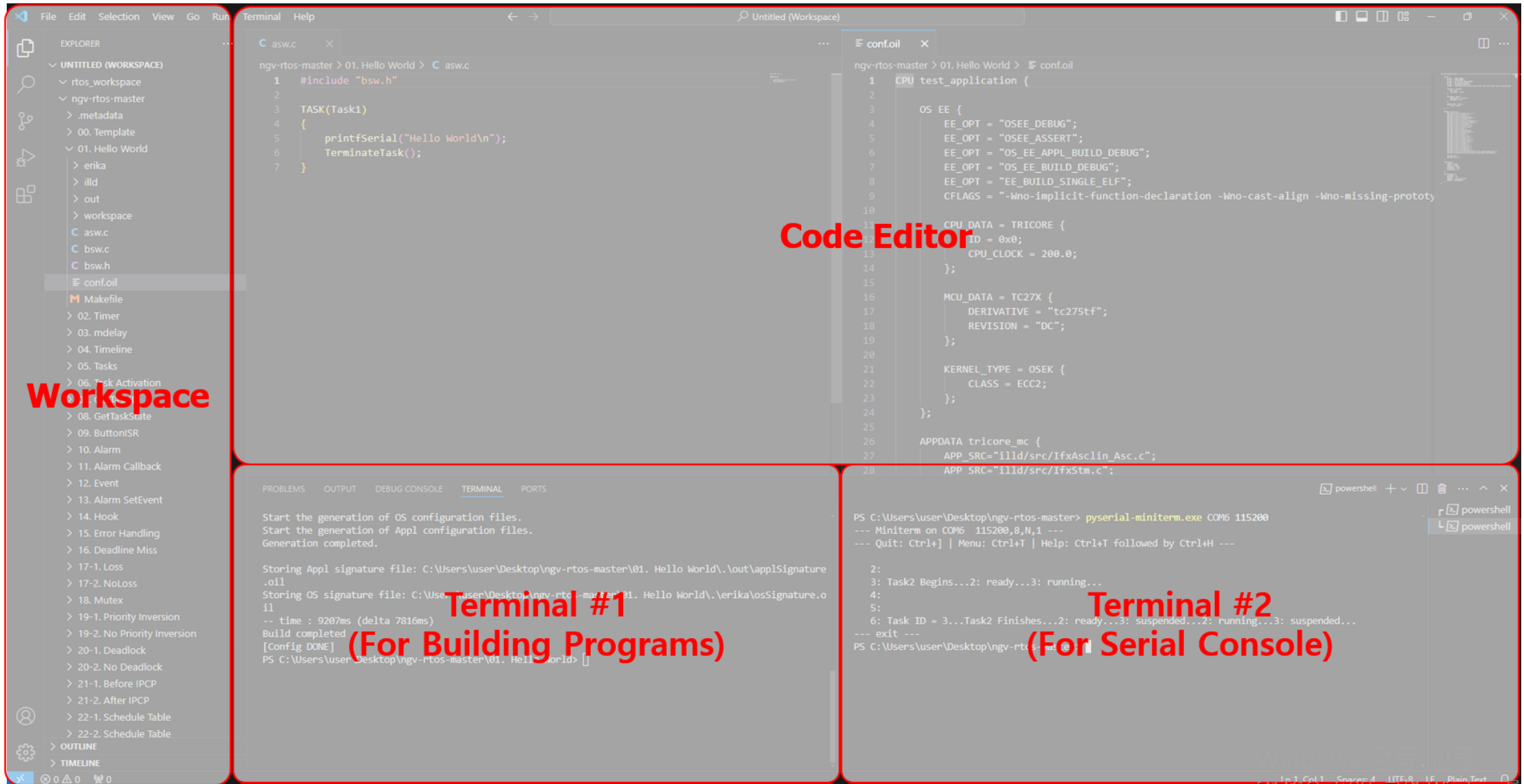
Readme
View license
155 stars
25 watching

Eclipse 기반 IDE (old)

- 프로젝트 생성
- OIL 파일, C/C++ 파일 편집
- 프로젝트 빌드
- 실행파일 다운로드
- ...

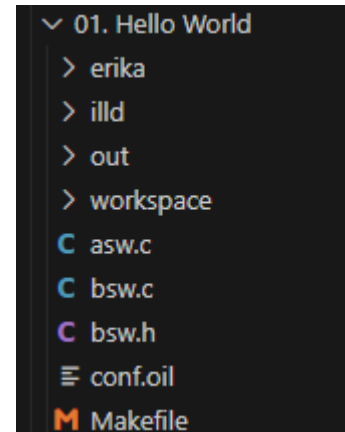


Visual Studio Code 기반 개발환경 (new)



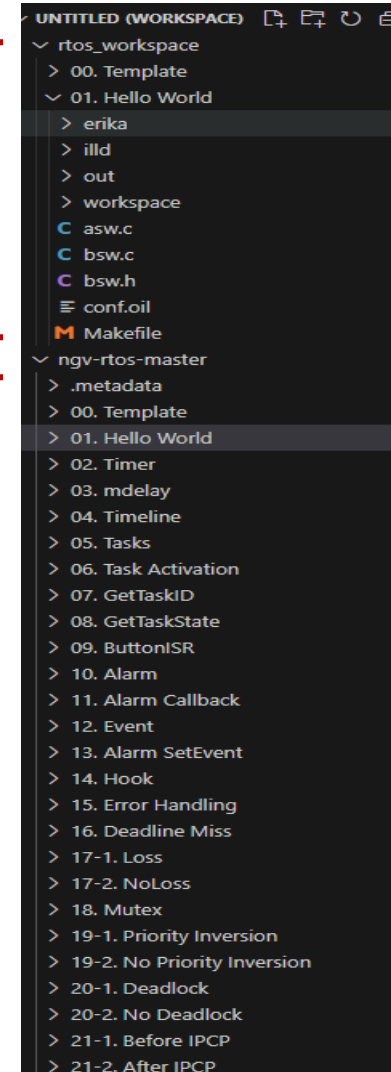
Workspace

- Directory Structure
 - rtos_workspace: working directory
 - ngv-rtos-master: example source files
- Example Projects Directory Structure
 - illd/: Infineon Low Level Driver
 - **asw.c**: Application SW code
 - bsw.c: Basic SW code
 - bsw.h: Basic SW header
 - **conf.oil**: OIL configuration file
 - Makefile: Top-level Makefile



Working

Examples



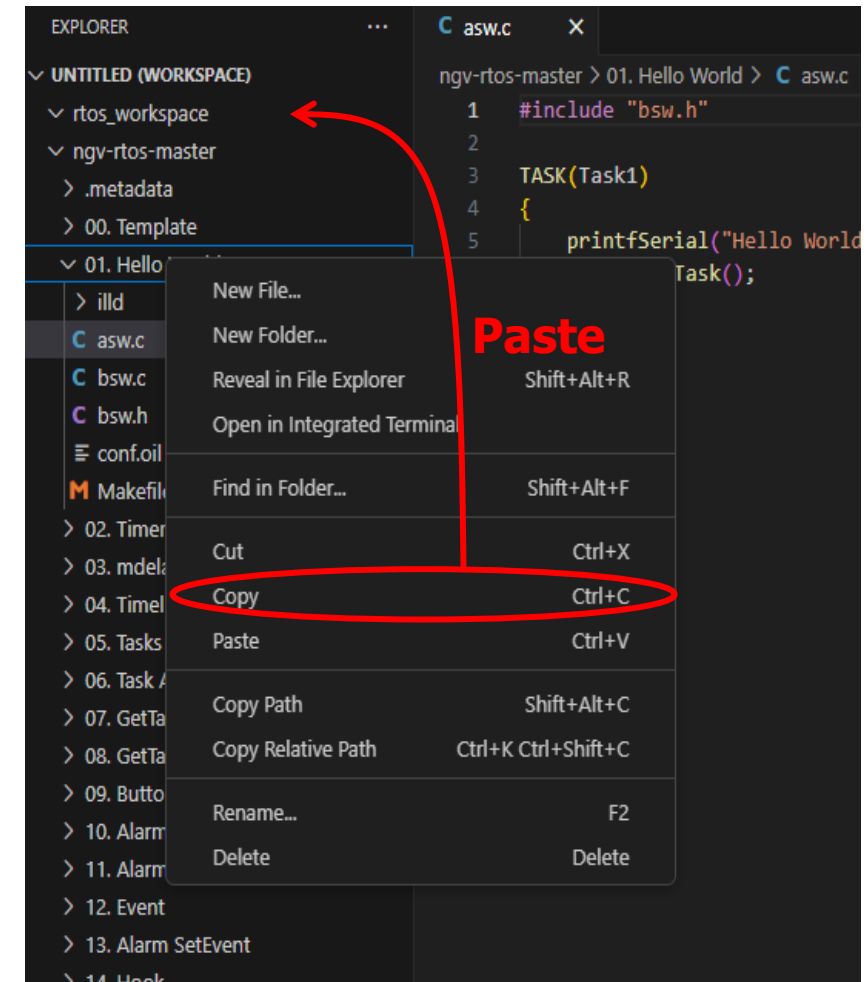
Workflow

- Basic Workflow

- 1) Copy a project (e.g., 01. Hello World) from to working directory
- 2) Edit source files (.C or .H) and OIL files
- 3) Build

- Build Process (in Terminal #1)

- 1) `cd 01. Hello World\`
- 2) `make config`
 - Generate OS kernel files from `conf.oil`
- 3) `make`
 - Generate an executable file

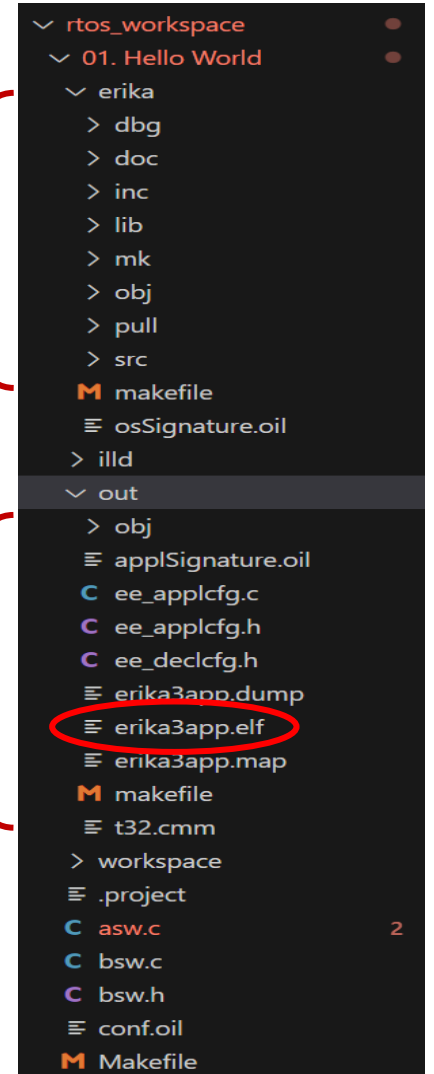


Generated Files

- erika/
 - OS source files
- out/
 - Generated files from the OIL file
 - Object files
 - Executable (ELF) file (erika3app.elf)

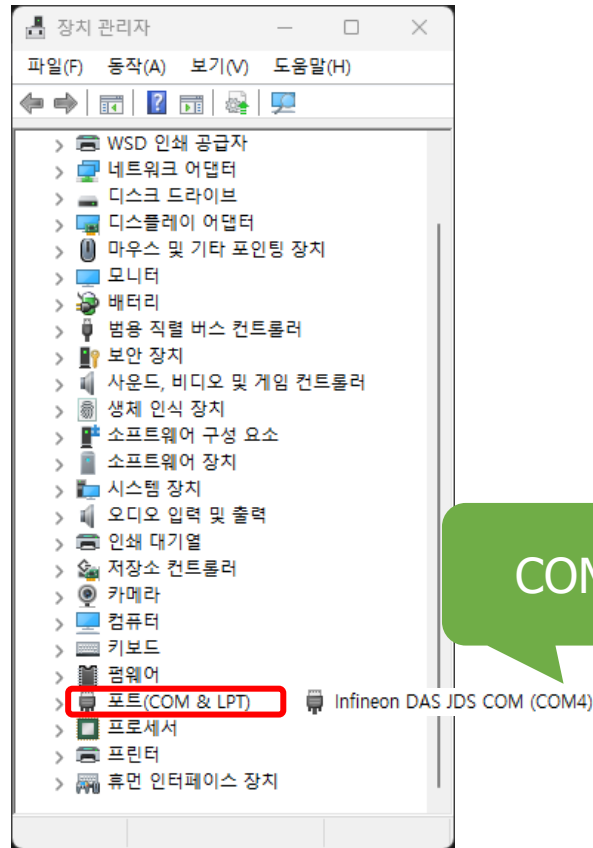
OS source files

Generated files



Serial Console

- 장치관리자에서 COM 포트 확인 (e.g., COM4)
- pyserial-miniterm 이용하여 시리얼 콘솔 시작 (Terminal #2)



COM 포트

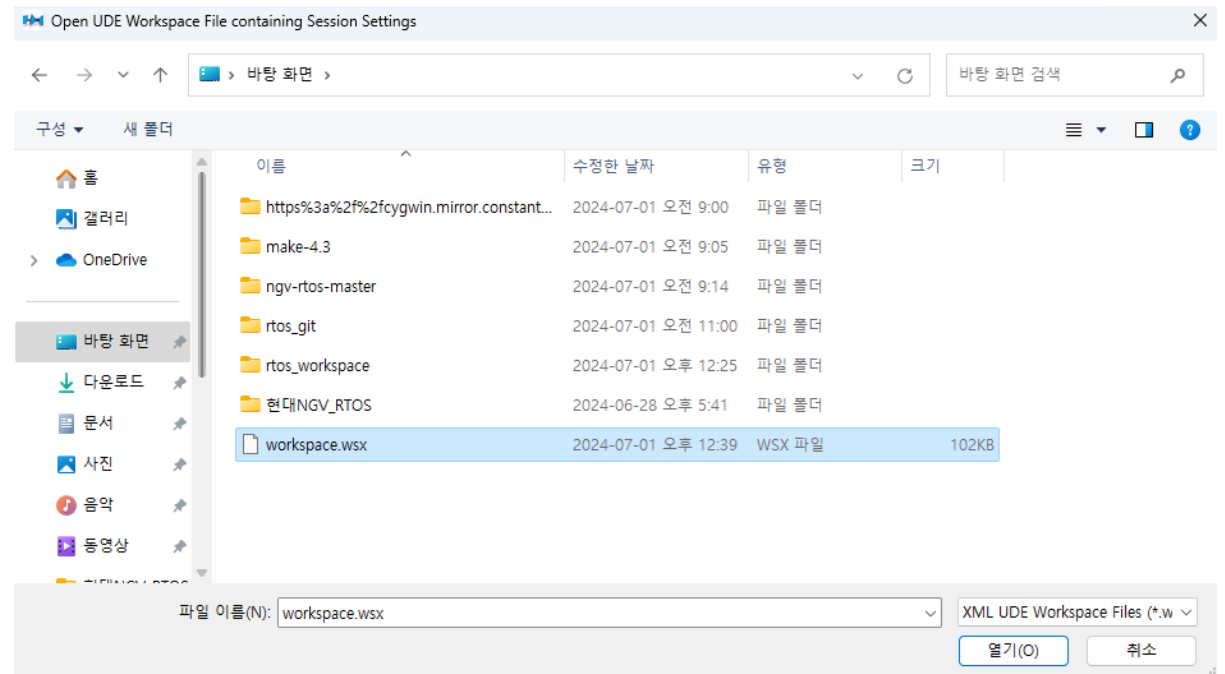
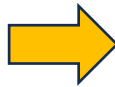
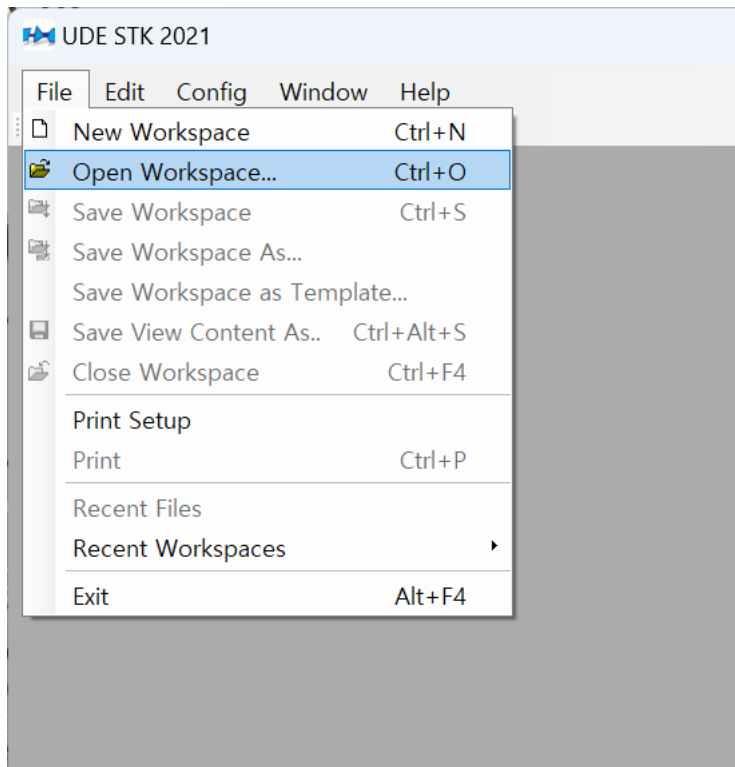
Baudrate

```
PS C:\Users\jongchank\work> pyserial-miniterm.exe COM4 115200
--- Miniterm on COM4 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---

```

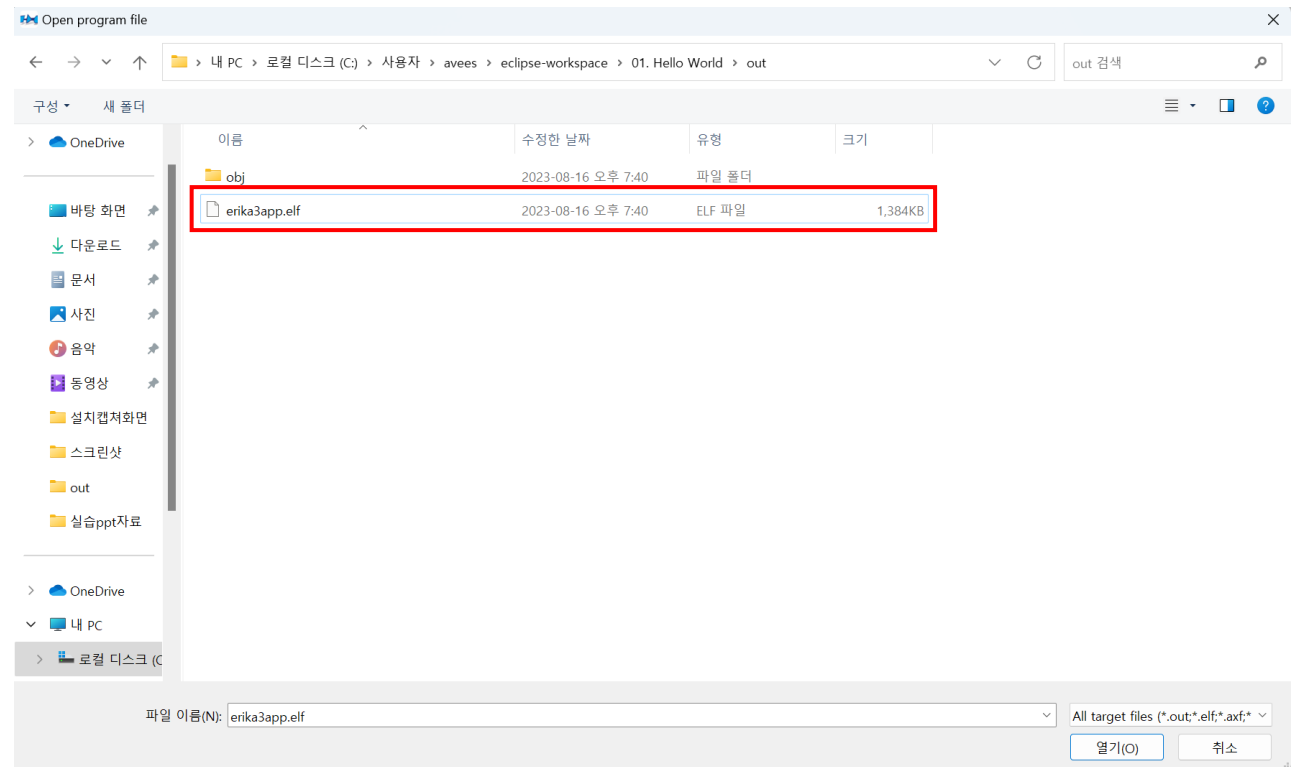
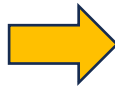
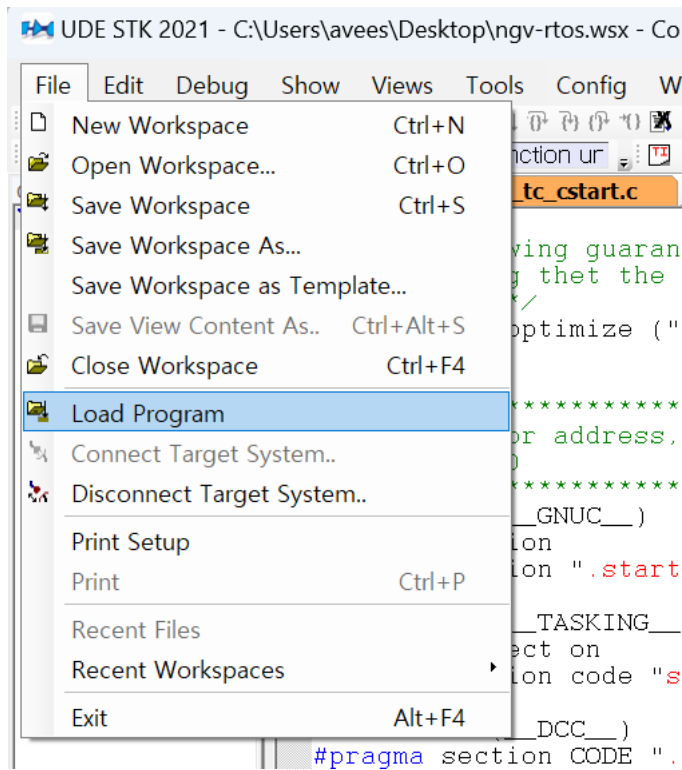

Program Flashing by UDE #1

- File → Open Workspace
- 바탕화면의 workspace.wsx 열기



Program Flashing by UDE #2

- File → Load Program
- 프로젝트 폴더의 out/erika3app.elf 열기

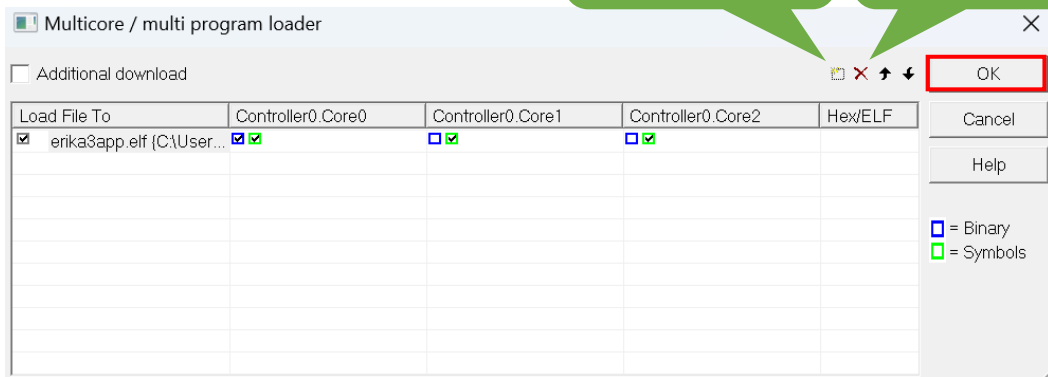


Program Flashing by UDE #3

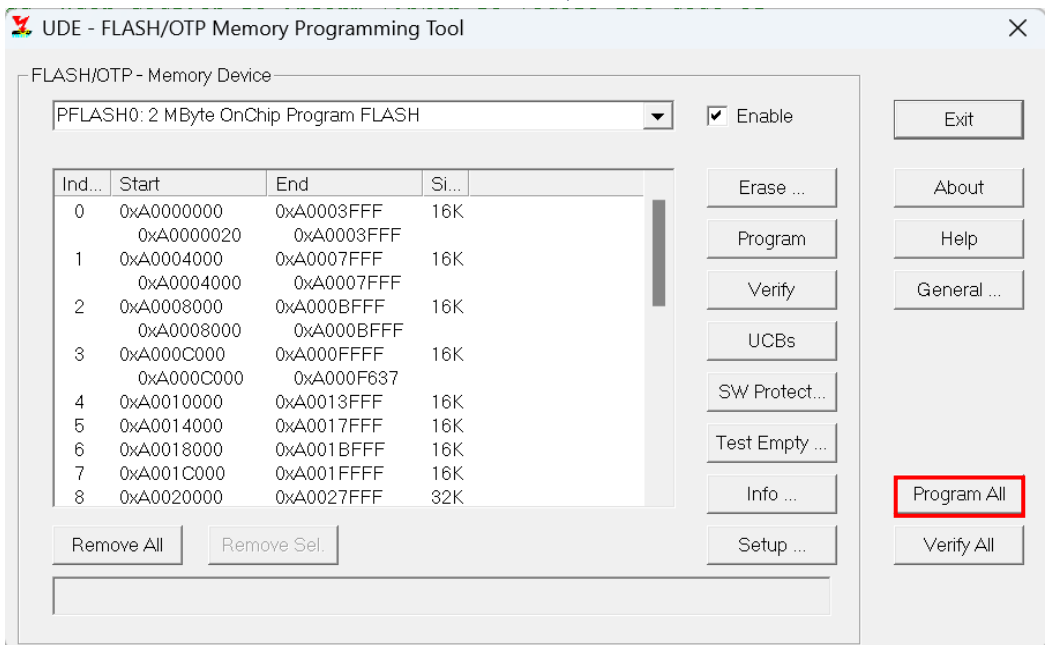
새 파일
열기

기존 파일
해제

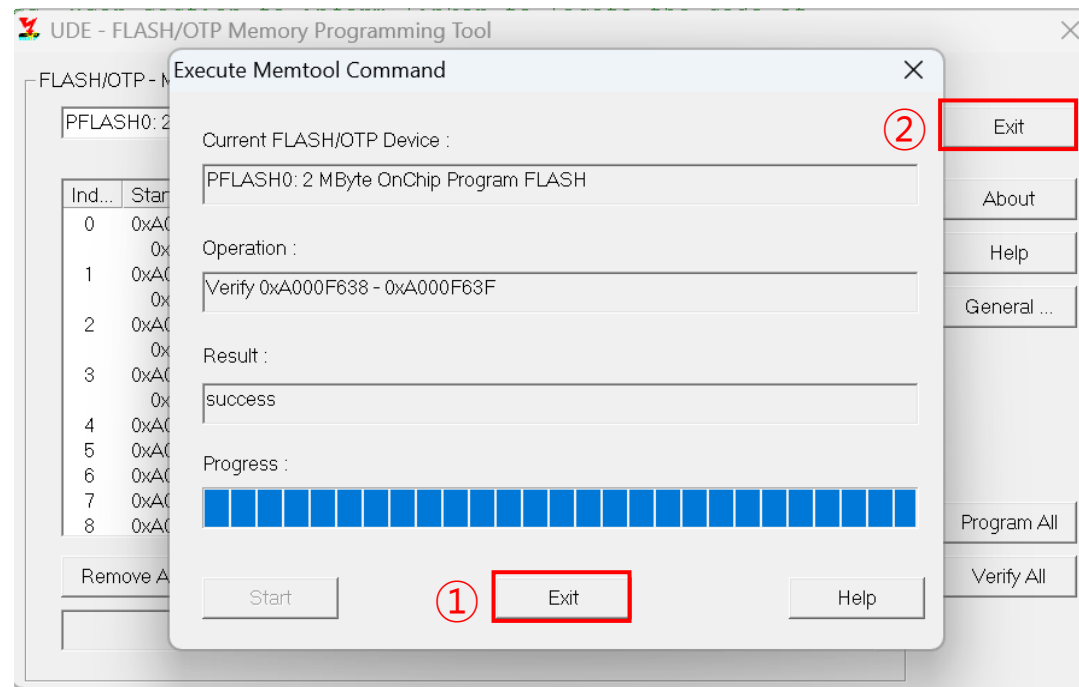
• Step1



• Step2

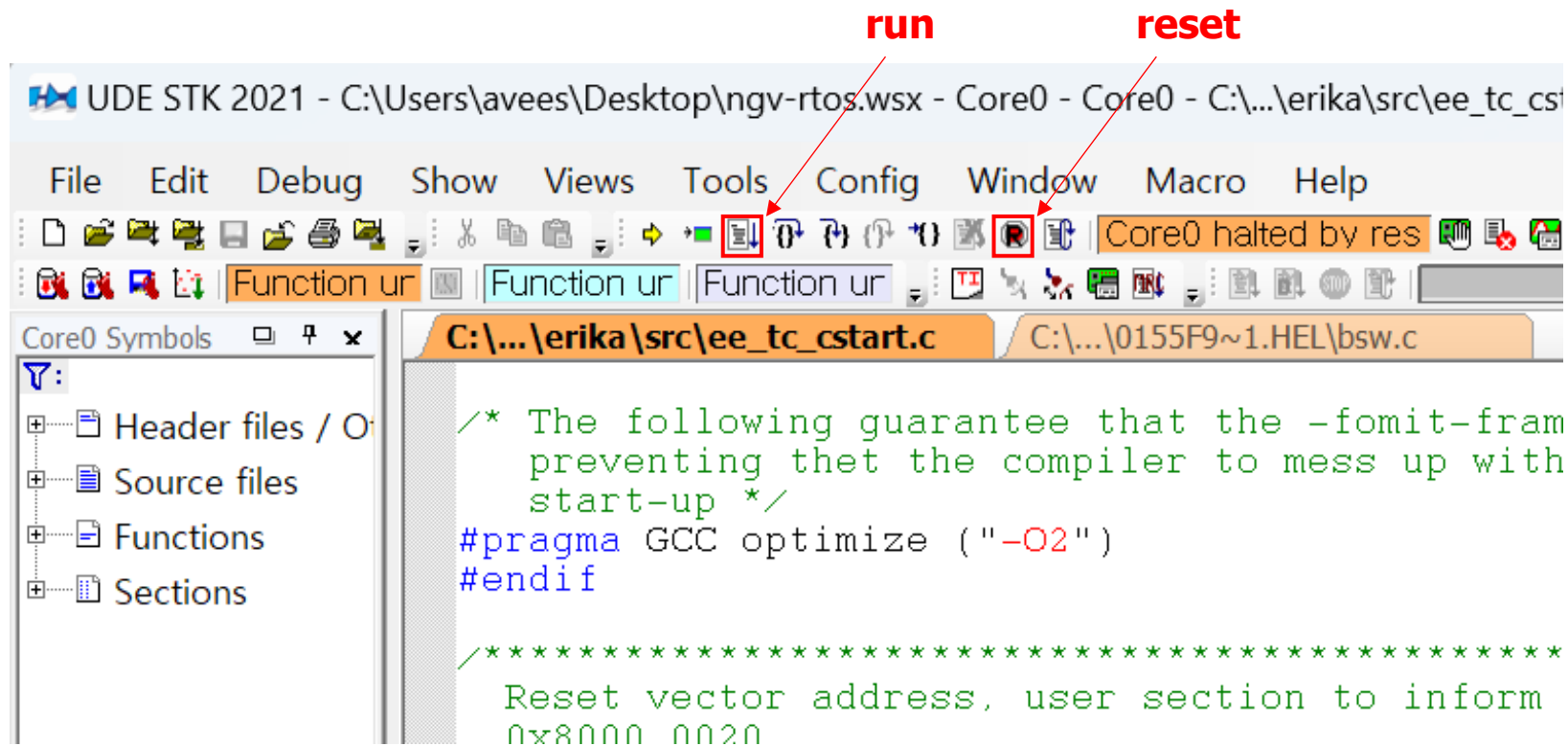


• Step3



Program Flashing by UDE #4

- 프로그램 실행 (run)
- 기존 프로그램 실행중일 경우 reset 후 실행



01. Hello World

- 00. Template 복사
- asw.c에 TASK 추가
- OIL 파일에 TASK 추가
- printfSerial() 함수 사용 (시리얼 콘솔 출력)

Make config는 OIL
변경시에만

```
$ make config  
$ make
```

```
#include "bsw.h"  
  
TASK(Task1)  
{  
    printfSerial("Hello World\n");  
  
    TerminateTask();  
}
```

```
TASK Task1 {  
    PRIORITY = 1;  
    STACK = SHARED;  
    SCHEDULE = FULL;  
    AUTOSTART = TRUE;  
    ACTIVATION = 1;  
};
```

자동
시작

01. Hello World

- OS 시작 후
- Hello World 출력

```
PS C:\Users\jongchank\work> pyserial-miniterm.exe COM4 115200
--- Miniterm on COM4 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---

.....
...OS Starts...
.....
Hello World
█
```

02. Timer

- C 파일에 ISR2로 TimerISR 추가
- OIL 파일에 TimerISR 추가
 - Category 2

1초 뒤
interrupt 등록

```
ISR2(TimerISR)
{
    osEE_tc_stm_set_sr0_next_match(1000000U);
    printfSerial("Timer\n");
}
```

```
ISR TimerISR {
    CATEGORY = 2;
    SOURCE = "STM0SR0";
    PRIORITY = 2;
};
```

02. Timer

- Hello World 출력후 Timer 반복

```
PS C:\Users\jongchank\work> pyserial-miniterm.exe COM4 115200
--- Miniterm on COM4 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---

.....
...OS Starts...
.....
Hello World
Timer
Timer
Timer
Timer
Timer
Timer
[]
```


03. mdelay

- mdelay 함수 이용 3초 실행시간

```
TASK(Task1)
{
    printfSerial("Hello World\n");

    mdelay(3000);

    printfSerial("Goodbye World\n");
    TerminateTask();
}
```

```
PS C:\Users\jongchank\work> pyserial-miniterm.exe COM4 115200
--- Miniterm on COM4 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---

.....
...OS Starts...
.....
Hello World
Goodbye World
[]
```

04. Timeline

- TimerISR 이용 초단위 Timeline 출력

```
ISR2(TimerISR)
{
    static long c = 0;
    osEE_tc_stm_set_sr0_next_match(1000000U);
    printfSerial("\n%4ld: ", c++);
}
```

```
.....
...OS Starts...
.....
Hello World

0:
1:
2: Goodbye World

3:
4:
5:
6:
7:
8:
9: []
```

05. Tasks

```
TASK(Task1)
{
    printfSerial("Task1 Begins...");
    mdelay(3000);
    printfSerial("Task1 Finishes...");

    TerminateTask();
}

TASK(Task2)
{
    printfSerial("Task2 Begins...");
    mdelay(3000);
    printfSerial("Task2 Finishes...");

    TerminateTask();
}
```

- 우선순위 2의 Task2 추가

클수록 높은
우선순위

```
TASK Task2 {
    PRIORITY = 2;
    STACK = SHARED;
    SCHEDULE = FULL;
    AUTOSTART = TRUE;
    ACTIVATION = 1;
};
```

05. Tasks

- Task2가 먼저 시작
- Task2 종료 후 Task1 시작
- 우선순위를 바꾼다면?

```
PS C:\Users\jongchank\work> pyserial-miniterm.exe COM4 115200
--- Miniterm on COM4 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---

.....
...OS Starts...
.....
Task2 Begins...
  0:
  1:
  2: Task2 Finishes...Task1 Begins...
  3:
  4:
  5: Task1 Finishes...
  6:
  7: []
```

06. Task Activation

```
ISR2(TimerISR)
{
    static long c = -4;
    osEE_tc_stm_set_sr0_next_match(1000000U);
    if (c == 0)
        ActivateTask(Task1);
    printfSerial("\n%4ld: ", c++);
}
```

```
TASK(Task1)
{
    printfSerial("Task1 Begins...");
    mdelay(3000);
    ActivateTask(Task2);
    mdelay(3000);
    printfSerial("Task1 Finishes...");
    TerminateTask();
}
```

```
TASK(Task2)
{
    printfSerial("Task2 Begins...");
    mdelay(3000);
    printfSerial("Task2 Finishes...");
    TerminateTask();
}
```

```
TASK Task1 {
    ...
    AUTOSTART = FALSE;
    ...
};
```

```
TASK Task2 {
    ...
    AUTOSTART = FALSE;
    ...
};
```

06. Task Activation

- Timeline -4부터 카운트다운
- Task2의 Task1 선점 확인
- ActivateTask 위치 바꾸면?
- 우선순위가 바뀌면?
- ChainTask 활용
- Task3까지 만들어서 연쇄 실행

```
-4:
-3:
-2:
-1:
 0: Task1 Begins...
 1:
 2:
 3: Task2 Begins...
 4:
 5:
 6: Task2 Finishes...
 7:
 8:
 9: Task1 Finishes...
10:
11: []
```

07. GetTaskID

```
TASK(Task1)
{
    TaskType id;
    printfSerial("Task1 Begins...");
    mdelay(3000);
    ActivateTask(Task2);
    mdelay(3000);
    GetTaskID(&id);
    printfSerial("Task ID = %d...", id);
    printfSerial("Task1 Finishes...");
    TerminateTask();
}
```

```
TASK(Task2)
{
    TaskType id;
    printfSerial("Task2 Begins...");
    mdelay(3000);
    GetTaskID(&id);
    printfSerial("Task ID = %d...", id);
    printfSerial("Task2 Finishes...");
    TerminateTask();
}
```

07. GetTaskID

- 자연수 Task ID 확인
- Unique ID일 뿐 정의된 의미 없음

```
-4:
-3:
-2:
-1:
0: Task1 Begins...
1:
2:
3: Task2 Begins...
4:
5:
6: Task ID = 3...Task2 Finishes...
7:
8:
9: Task ID = 2...Task1 Finishes...
10:
11: □
```

Task2의
ID

Task1의
ID

08. GetTaskState

```
TASK(TaskM)
{
    printState(Task1);
    printState(Task2);

    TerminateTask();
}
```

```
TASK TaskM {
    PRIORITY = 3;
    STACK = SHARED;
    SCHEDULE = FULL;
    AUTOSTART = FALSE;
    ACTIVATION = 1;
};
```

```
void printState(TaskType id) {
    TaskStateType state;

    if (GetTaskState(id, &state) == E_OK) {
        switch (state) {
            case SUSPENDED:
                printfSerial("%d: suspended...", id);
                break;
            case READY:
                printfSerial("%d: ready...", id);
                break;
            case WAITING:
                printfSerial("%d: waiting...", id);
                break;
            case RUNNING:
                printfSerial("%d: running...", id);
                break;
        }
    }
}
```

08. GetTaskState

```
TASK(Task1)
{
    TaskType id;
    printfSerial("Task1 Begins...");
    printState(Task1);
    printState(Task2);
    mdelay(3000);
    ActivateTask(Task2);
    printState(Task1);
    printState(Task2);
    mdelay(3000);
    GetTaskID(&id);
    printfSerial("Task ID = %d...", id);
    printfSerial("Task1 Finishes...");
    ChainTask(TaskM);
}
```

```
TASK(Task2)
{
    TaskType id;
    printfSerial("Task2 Begins...");
    printState(Task1);
    printState(Task2);
    mdelay(3000);
    GetTaskID(&id);
    printfSerial("Task ID = %d...", id);
    printfSerial("Task2 Finishes...");
    ChainTask(TaskM);
}
```

08. GetTaskState

- Task 상태 변화 관찰
- 우선순위, Activation 패턴 변화의 영향은?

```
-1:
0: Task1 Begins...2: running...4: suspended...
1:
2:
3: Task2 Begins...2: ready...4: running...
4:
5:
6: Task ID = 4...Task2 Finishes...2: ready...4: suspended...2: running...4: suspend
ed...
7:
8:
9: Task ID = 2...Task1 Finishes...2: suspended...4: suspended...
10:
11:
12:
13:
14: □
```

09. ButtonISR

- ButtonISR 추가 (Button 입력에 반응하여 Task Activate)

```
ISR2(ButtonISR)
{
    unsigned int a0;
    DisableAllInterrupts();
    osEE_tc_delay(5000);
    a0 = readADCValue(3);
    if (a0 < 500) { /* TOP */
        printfSerial("<BUTTON:T>");
        ActivateTask(Task1);
    } else if (a0 < 1200) { /* DOWN */
        printfSerial("<BUTTON:D>");
        ActivateTask(Task2);
    } else if (a0 < 1600) { /* LEFT */
        printfSerial("<BUTTON:L>");
    } else if (a0 < 2200) { /* RIGHT */
        printfSerial("<BUTTON:R>");
    } else {
        printfSerial("<BUTTON:?>");
    }
    osEE_tc_delay(3000);
    EnableAllInterrupts();
}
```

```
ISR ButtonISR {
    CATEGORY = 2;
    SOURCE = "SCUERU0";
    PRIORITY = 10;
};
```

09. ButtonISR

- Task1, Task2 원상복귀

```
TASK(Task1)
{
    printfSerial("Task1 Begins...");
    mdelay(3000);
    printfSerial("Task1 Finishes...");
    TerminateTask();
}
```

```
TASK(Task2)
{
    printfSerial("Task2 Begins...");
    mdelay(3000);
    printfSerial("Task2 Finishes...");
    TerminateTask();
}
```

09. ButtonISR

- 중복 Activation 하려면?
 - ACTIVATION = 1; 수정 필요
- ButtonISR에서 30초 mdelay 실행하면?
- ButtonISR과 TimerISR의 우선순위에 따른 변화?

10. Alarm

- OIL에 COUNTER와 ALARM 추가

```
COUNTER mycounter {  
    MINCYCLE = 1;  
    MAXALLOWEDVALUE = 127;  
    TICKSPERBASE = 1;  
};
```

```
ALARM alarm1 {  
    COUNTER = mycounter;  
    ACTION = ACTIVATETASK {  
        TASK = Task1;  
    };  
    AUTOSTART = TRUE {  
        ALARMTIME = 5;  
        CYCLETIME = 10;  
    };  
};
```

```
ALARM alarm2 {  
    COUNTER = mycounter;  
    ACTION = ACTIVATETASK {  
        TASK = Task2;  
    };  
    AUTOSTART = TRUE {  
        ALARMTIME = 5;  
        CYCLETIME = 20;  
    };  
};
```

10. Alarm

- TimerISR에서
 - ActivateTask(Task1) 삭제
 - mycounter 증가

```
ISR2(TimerISR)
{
    static long c = -4;
    osEE_tc_stm_set_sr0_next_match(1000000U);
if (c == 0)
    ActivateTask(Task1);
    IncrementCounter(mycounter);
    printfSerial("\n%4ld: ", c++);
}
```


11. Alarm Callback

- 콜백 함수 등록

```
ALARMCALLBACK(MyCallback)
{
    printfSerial("<MyCallback>");
}
```

```
ALARM alarm3 {
    COUNTER = mycounter;
    ACTION = ALARMCALLBACK {
        ALARMCALLBACKNAME = "MyCallback";
    };
    AUTOSTART = TRUE {
        ALARMTIME = 5;
        CYCLETIME = 15;
    };
};
```

12. Event

```
ISR2(ButtonISR)
{
    unsigned int a0;
    DisableAllInterrupts();
    osEE_tc_delay(5000);
    a0 = readADCValue(3);
    if (a0 < 500) {
        printfSerial("<BUTTON:T>");
        SetEvent(Task2, Event1);
    } else if (a0 < 1200) {
        printfSerial("<BUTTON:D>");
        SetEvent(Task2, Event2);
    } else if (a0 < 1600) {
        printfSerial("<BUTTON:L>");
    } else if (a0 < 2200) {
        printfSerial("<BUTTON:R>");
    } else {
        printfSerial("<BUTTON:?>");
    }
    osEE_tc_delay(3000);
    EnableAllInterrupts();
}
```

```
CPU_DATA = TRICORE {
    ID = 0x0;
    CPU_CLOCK = 200.0
    MULTI_STACK = TRUE;
};
...
EVENT Event1 { MASK = AUTO; };
EVENT Event2 { MASK = AUTO; };
...
TASK Task2 {
    PRIORITY = 2;
    STACK = PRIVATE {
        SIZE = 1024;
    };
    SCHEDULE = FULL;
...
    EVENT = Event1;
    EVENT = Event2;
...
}
```

12. Event

```
TASK(Task2)
{
    EventMaskType mask;
    printfSerial("Task2 Begins...");
    printfSerial("Task2 Waits...");
    WaitEvent(Event1 | Event2);
    printfSerial("Task2 Wakes Up...");
    GetEvent(Task2, &mask);
    if (mask & Event1) {
        printfSerial("[Event1]");
        ClearEvent(Event1);
    }
    if (mask & Event2) {
        printfSerial("[Event2]");
        ClearEvent(Event2);
    }
    printfSerial("Task2 Finishes...");
    TerminateTask();
}
```

12. Event

- ClearEvent는 왜 필요한가?
- 우선순위 반대의 경우 스케줄링

13. Alarm SetEvent

- Alarm을 이용한 주기적인 SetEvent Action

```
ALARM alarm3 {  
    COUNTER = mycounter;  
    ACTION = SETEVENT {  
        TASK = Task2;  
        EVENT = Event1;  
    };  
    AUTOSTART = TRUE {  
        ALARMTIME = 7;  
        CYCLETIME = 20;  
    };  
};
```

14. Hook

- OIL Hook 사용 설정

```
KERNEL_TYPE = OSEK {  
    CLASS = ECC2; // Default  
};
```

```
STARTUPHOOK = TRUE;  
SHUTDOWNHOOK = TRUE;  
PRETASKHOOK = TRUE;  
POSTTASKHOOK = TRUE;  
};
```

14. Hook

- Task2 원상복귀

```
TASK(Task2)
{
    printfSerial("Task2 Begins...");
    mdelay(3000);
    printfSerial("Task2 Finishes...");
    TerminateTask();
}
```

- ButtonISR에 ShutdownOS 추가

```
ISR2(ButtonISR)
{
    ...
    if (a0 < 500) {
        printfSerial("<BUTTON:T>");
        SetEvent(Task2, Event1);
    } else if (a0 < 1200) {
        printfSerial("<BUTTON:D>");
        SetEvent(Task2, Event2);
    } else if (a0 < 1600) {
        printfSerial("<BUTTON:L>");
    } else if (a0 < 2200) {
        printfSerial("<BUTTON:R>");
        ShutdownOS(1);
    }
    ...
}
```

14. Hook

- StartupHook
- ShutdownHook

```
void StartupHook(void)
{
    printfSerial("[StartupHook]");
}

void ShutdownHook(StatusType Error)
{
    printfSerial("[ShutdownHook]");
}
```


14. Hook

- PreTaskHook
- PostTaskHook

```
void PreTaskHook(void)
{
    TaskType id;
    GetTaskID(&id);
    printfSerial("[PreTaskHook(%d)]", id);
    printState(Task1);
    printState(Task2);
}

void PostTaskHook(void)
{
    TaskType id;
    GetTaskID(&id);
    printfSerial("[PostTaskHook(%d)]", id);
    printState(Task1);
    printState(Task2);
}
```

15. Error Handling

- OIL 파일 설정

오류가 발생한 Service ID와
Parameter 정보 접근

```
...  
    KERNEL_TYPE = OSEK {  
        CLASS = ECC2; // Default  
    };  
    STARTUPHOOK = FALSE;  
    SHUTDOWNHOOK = FALSE;  
    PRETASKHOOK = FALSE;  
    POSTTASKHOOK = FALSE;  
    ERRORHOOK = TRUE;  
    USEGETSERVICEID = TRUE;  
    USEPARAMETERACCESS = TRUE;  
};  
...
```

15. Error Handling

```
ISR2(TimerISR)
{
    static long c = -4;
    osEE_tc_stm_set_sr0_next_match(1000000U);
    TaskStateType s;
    if (c == 5) {
        GetTaskState(30, &s);
    }
    IncrementCounter(mycounter);
    printfSerial("\n%4ld: ", c++);
}

void ErrorHook(StatusType error)
{
    printfSerial("[ErrorHook: error = %d, service = %d, TaskID = %d]",
        error,
        OSErrorGetServiceId(),
        OSError_GetTaskState_TaskID());
}
```

고의 에러
잘못된 ID = 30

Parameter 정보 접근 매크로
(ee_oo_api_osek.h)

15. Error Handling

```
E_OK,                /* ((StatusType)0) */
E_OS_ACCESS,         /* ((StatusType)1) */
E_OS_CALLEVEL,       /* ((StatusType)2) */
E_OS_ID,             /* ((StatusType)3) */
E_OS_LIMIT,          /* ((StatusType)4) */
E_OS_NOFUNC,         /* ((StatusType)5) */
E_OS_RESOURCE,       /* ((StatusType)6) */
E_OS_STATE,          /* ((StatusType)7) */
E_OS_VALUE,          /* ((StatusType)8) */
E_OS_SERVICEID,      /* ((StatusType)9) */
E_OS_ILLEGAL_ADDRESS, /* ((StatusType)10) */
...
```

ee_api_types.h

```
OSServiceId_ActivateTask      = (0),
OSServiceId_TerminateTask     = (2),
OSServiceId_ChainTask         = (4),
OSServiceId_Schedule          = (6),
OSServiceId_GetTaskID         = (8),
OSServiceId_GetTaskState      = (10)
,
OSServiceId_DisableAllInterrupts = (12)
,
OSServiceId_EnableAllInterrupts = (14)
,
OSServiceId_SuspendAllInterrupts = (16)
,
OSServiceId_ResumeAllInterrupts = (18)
,
OSServiceId_SuspendOSInterrupts = (20)
,
OSServiceId_ResumeOSInterrupts = (22)
,
...
```

15. Error Handling

```
/**
 * \brief This macro returns the TaskID parameter passed to ActivateTask().
 * \ingroup primitives-hook
 */
#define OSErrors_ActivateTask_TaskID()\
    ((TaskType)osEE_get_api_param1().num_param)

/**
 * \brief This macro returns the TaskID parameter passed to ChainTask().
 * \ingroup primitives-hook
 */
#define OSErrors_ChainTask_TaskID()\
    ((TaskType)osEE_get_api_param1().num_param)

/**
 * \brief This macro returns the TaskID parameter passed to GetTaskID().
 * \ingroup primitives-hook
 */
#define OSErrors_GetTaskID_TaskID()\
    ((TaskRefType)osEE_get_api_param1().p_param)
```

ee_oo_api_osek.h

16. Deadline Miss

```
ISR2(TimerISR)
{
    static long c = -4;
    osEE_tc_stm_set_sr0_next_match(1000000U
);
    IncrementCounter(mycounter);
    printfSerial("\n%4ld: ", c++);
}

TASK(Task1)
{
    TaskType id;
    printfSerial("Task1 Begins...");
    mdelay(7000);
    printfSerial("Task1 Finishes...");
    TerminateTask();
}
```

```
.....
...OS Starts...
.....

-4:
-3:
-2:
-1:
0: Task2 Begins...
1:
2:
3: Task2 Finishes...Task1 Begins...
4:
5:
6:
7:
8: Kernel Internal Error
9: [ErrorHook: error = 4, service = 82 TaskID = -1]
10: Task1 Finishes...
11: 이 경우 Task1의 Deadline Miss
12: : Activation 수 초과
13:
```

17-1. Loss (공유 자원 data loss 확인)

- Task1: Low priority, AUTOSTART
- Task2: High priority, 1 ms 주기 실행
- 공유 전역 변수 (volatile unsigned long shared)
 - Task1의 루프에서 shared++
 - Task2 주기적으로 반복 실행하며 shared++
- 양 쪽에서 더한 숫자가 모두 유지되는지 확인
- Task와 ISR 사이에서도 같은 문제가 발생하는지 확인

17-1. Loss

```
int main(void)
{
    osEE_tc_stm_set_clockpersec();
    osEE_tc_stm_set_sr0(1000U, 1U);
```

bsw.c

```
#include "bsw.h"
volatile unsigned long shared = 0;
```

```
TASK(Task1)
{
    unsigned long i;
    printfSerial("Task1 Begins...\n");
    for (i = 0; i < 20000000; i++) {
        shared++;
    }
    printfSerial("Added 20000000 to shared\n");
    printfSerial("counter = %lu\n", shared);
    printfSerial("Task1 Finishes...\n");
    TerminateTask();
}
```

```
TASK(Task2)
{
    static unsigned long i = 0;
    if (i < 500) {
        shared++;
    } else if (i == 500) {
        printfSerial("Added 500 to shared\n");
    }
    i++;
    TerminateTask();
}
```


17-1. Loss

- Resource를 이용해서 Integrity Loss 문제 해결 필요

```
.....  
...OS Starts...  
.....  
Task1 Begins...  
Added 500 to shared  
Added 20000000 to shared  
counter = 20000256  
Task1 Finishes...  
█
```

20000000 + 500 = 20000256(!!!)

17-2. No Loss

- OSEK의 RESOURCE 기능을 이용하여 Data Loss 문제 해결

```
GetResource(S1);  
shared++;  
ReleaseResource(S1);
```

```
RESOURCE S1 {  
    RESOURCEPROPERTY = STANDARD;  
};
```

...

```
TASK Task1 {  
    PRIORITY = 1;  
    STACK = SHARED;  
    SCHEDULE = FULL;  
    AUTOSTART = TRUE;  
    ACTIVATION = 1;  
    RESOURCE = S1;  
};
```

17-2. No Loss

- Data Integrity 문제 해결

```
.....  
...OS Starts...  
.....  
Task1 Begins...  
Added 500 to shared  
Added 20000000 to shared  
counter = 20000500  
Task1 Finishes...
```

$20000000 + 500 = 20000500(!!!)$

18. Mutex

Waiting/Wakeup 을 위
해 Event 지정 필요

• mutex.c

```
#include "ee.h"
#include "bsw.h"
#include "mutex.h"
```

```
void InitMutex(MutexType *mutex, EventMaskType event)
{
    mutex->flag = UNLOCKED;
    mutex->waiting_task = 0;
    mutex->event = event;
}
```

```
void GetMutex(MutexType *mutex)
{
    if (mutex->flag == LOCKED) {
        printfSerial("  --> BLock");
        GetTaskID(&(mutex->waiting_task));
        WaitEvent(mutex->event);
    }
    mutex->flag = LOCKED;
}
```

```
void ReleaseMutex(MutexType *mutex)
{
    if (mutex->flag == LOCKED) {
        mutex->flag = UNLOCKED;
        if (mutex->waiting_task != 0) {
            SetEvent(mutex->waiting_task, mutex->event);
        }
    }
}
```

**PCP 없이 Mutex 사용할 경우 문제점을
확인하기 위한 Dummy 구현**

18. Mutex

- mutex.h

```
#ifndef MUTEX_H_
#define MUTEX_H_

#define LOCKED    1
#define UNLOCKED  0

typedef struct _MutexType {
    int flag;
    EventMaskType event;
    TaskType waiting_task;
} MutexType;

void InitMutex(MutexType *mutex, EventMaskType event);

void GetMutex(MutexType *mutex);

void ReleaseMutex(MutexType *mutex);

#endif /* MUTEX_H_ */
```

18. Mutex

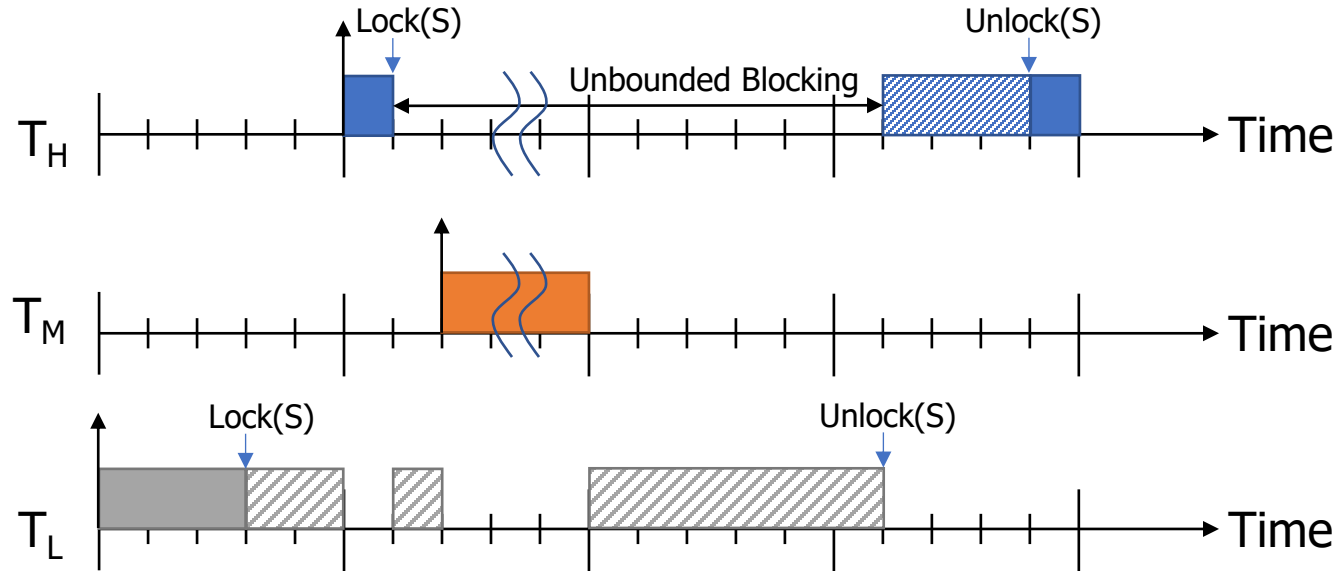
- Mutex 선언
- Timer ISR 이용
 - Mutex 초기화
 - Task Activation
- Mutex의 동작 확인

```
MutexType s1;

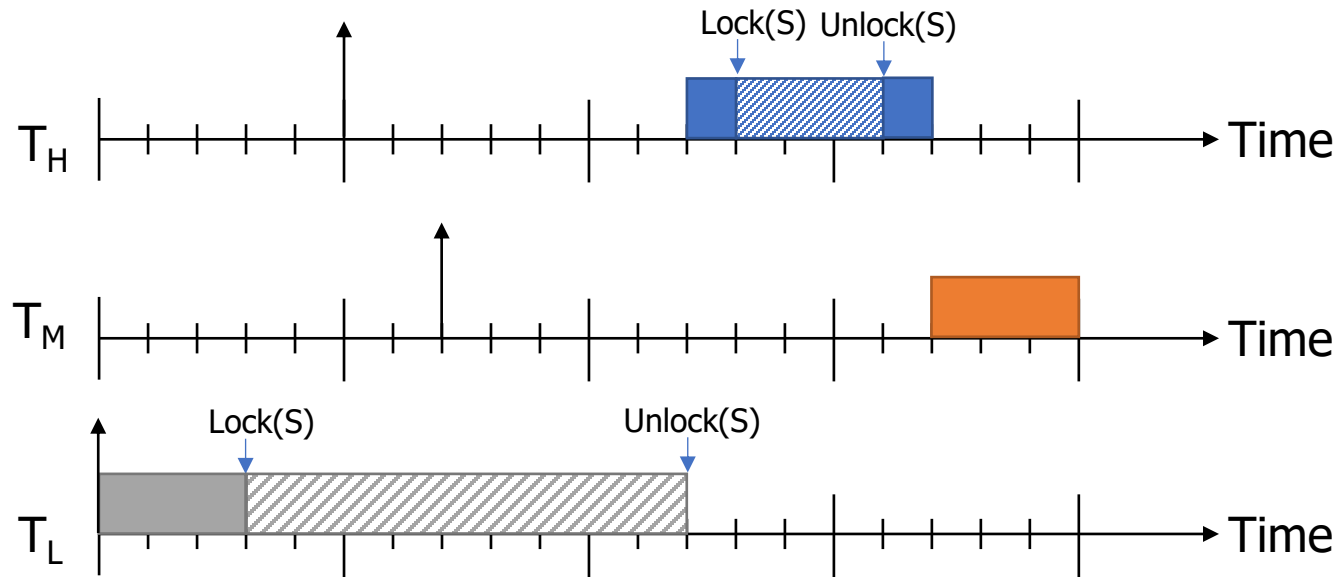
ISR2(TimerISR)
{
    osEE_tc_stm_set_sr0_next_match(1000000U);
    static long c = -5;
    printfSerial("\n%4ld: ", ++c);
    if(c == -4) {
        InitMutex(&s1, Event1);
    } else if (c == 0) {
        ActivateTask(TaskL);
    } else if (c == 5) {
        ActivateTask(TaskH);
    }
}
```

19-1. Priority Inversion

- 아래 스케줄 재현하기
- PCP가 적용된 RESOURCE를 이용하여 스케줄 변화 확인

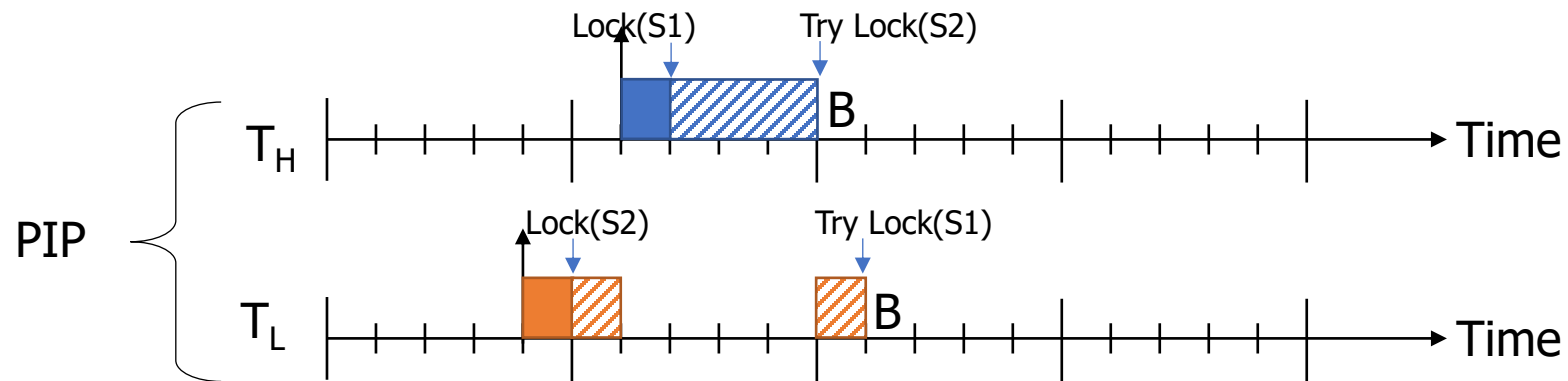


19-2. No Priority Inversion

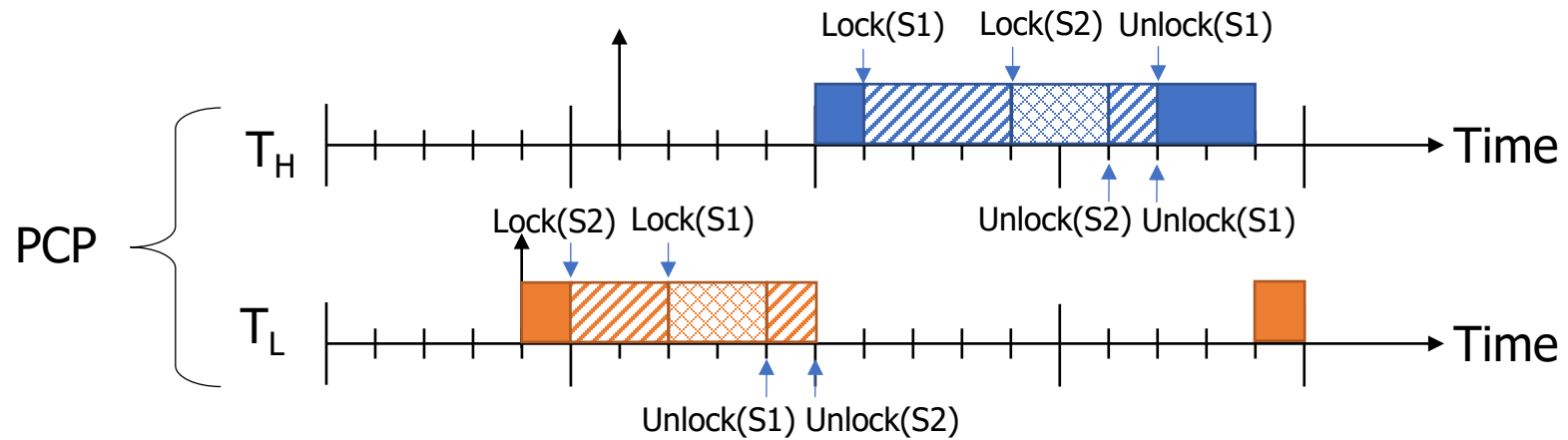


20-1. Deadlock

- Deadlock 재현하기
- PCP가 적용된 RESOURCE를 이용하여 Deadlock 해결

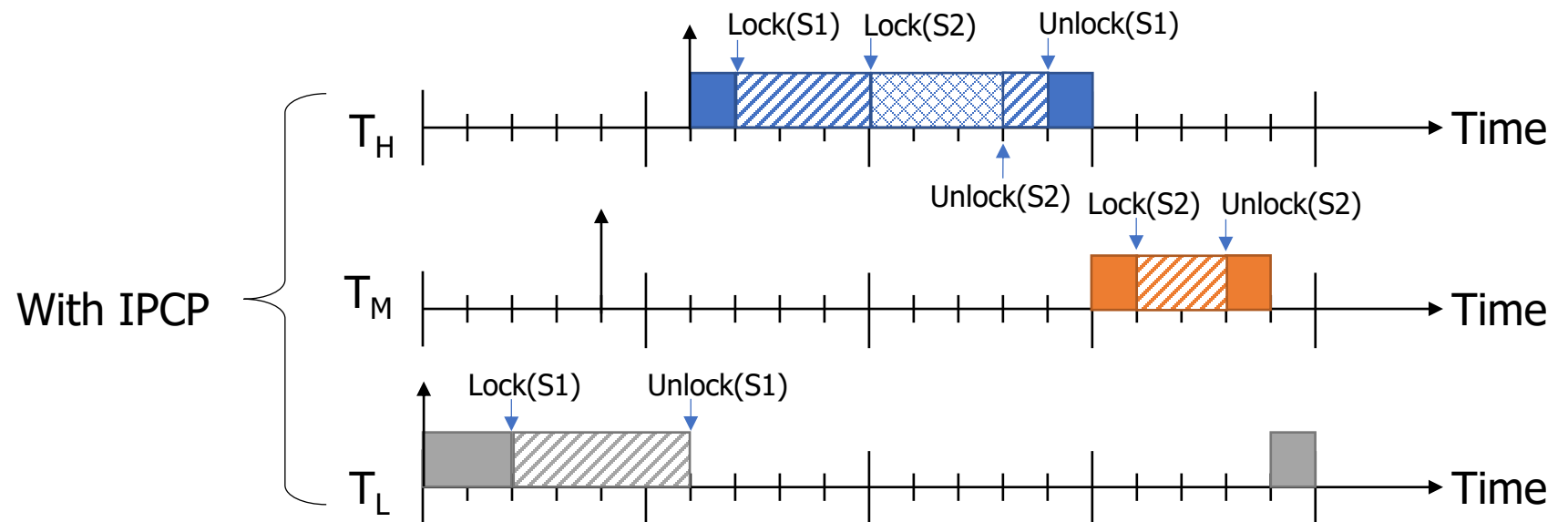
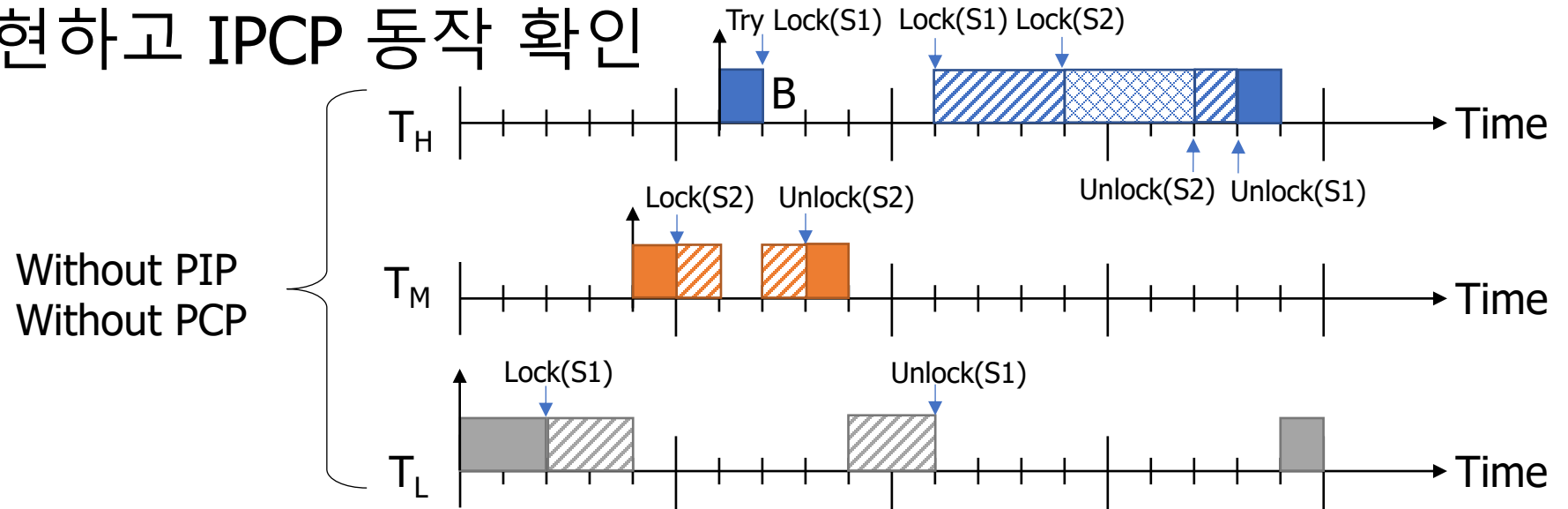


20-2. No Deadlock



21. Before/After IPCP

- 아래 스케줄 재현하고 IPCP 동작 확인



Cooperative Scheduling

- Non-preemptive 에서 Scheduling Point 설정

```
TASK(Task1)
{
    ...
    Schedule();
    ...
    Schedule();
    ...
}
```

Scheduling Points

- Preemptive 에서 Scheduling Disable

```
TASK(Task1)
{
    ...
    GetResource(RES_SCHEDULER);
    ...
    ReleaseResource(RES_SCHEDULER);
    ...
}
```

No scheduling allowed

Alarm 기반 Activation의 문제점

- OSEK에서 Periodic Task 작성 방법
 - Counter와 연결된 Alarm에서 Activate하도록 OIL 설정
- 문제점
 - Alarm은 런타임에 취소/변경이 가능
 - 실수 혹은 악의적으로 Alarm 변경시 Periodic Task 오동작 가능성
- Example)
 - CancelAlarm() 호출

13.6.3.5 CancelAlarm

Syntax: StatusType CancelAlarm (AlarmType <AlarmID>)

Parameter (In):
 AlarmID Reference to an alarm

Parameter (Out): none

Description: The system service cancels the alarm <AlarmID>.

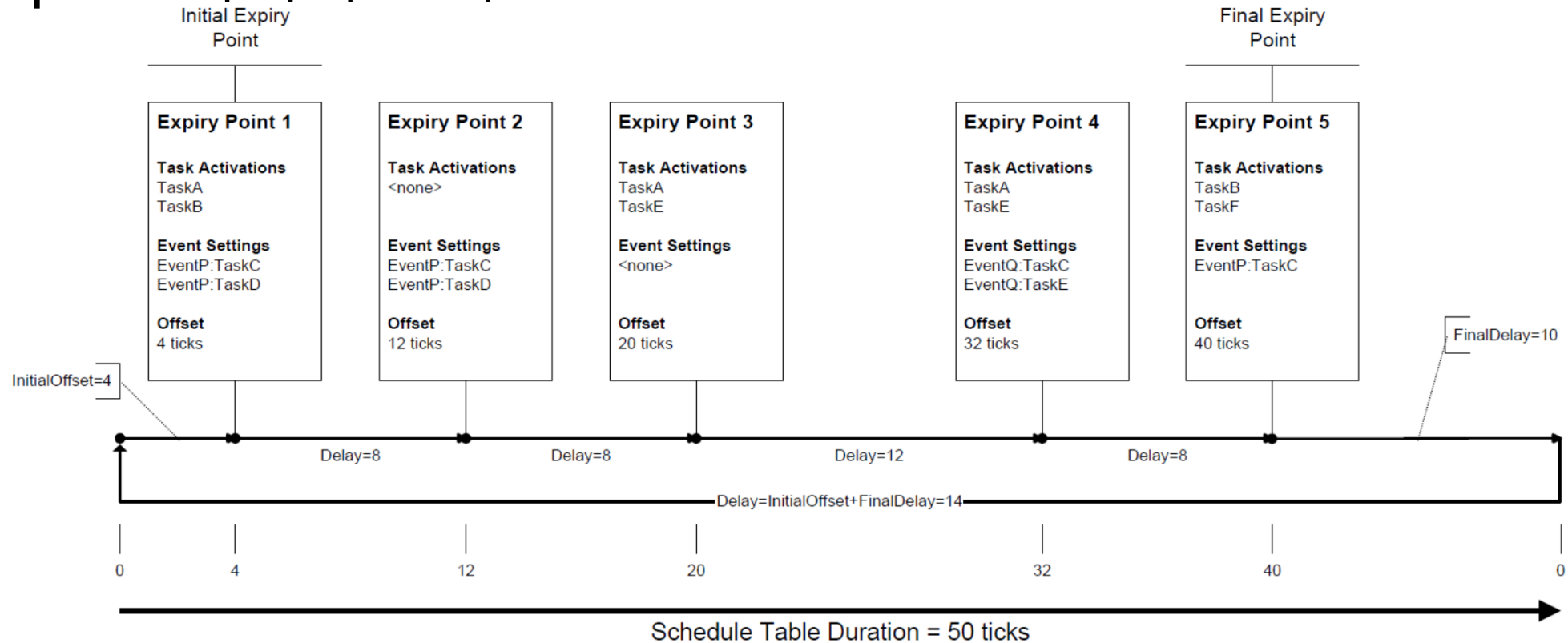
Particularities: Allowed on task level and in ISR, but not in hook routines.
Status:

- Standard: • No error, E_OK
 • Alarm <AlarmID> not in use, E_OS_NOFUNC
- Extended: • Alarm <AlarmID> is invalid, E_OS_ID

Conformance: BCC1, BCC2, ECC1, ECC2

Schedule Table Concepts

- Duration: Schedule table의 사이클 타임
- Expiry points: Duration 안에서의 상대 시간 (Activate, SetEvent 가능)
- Initial Offset: 첫 expiry point
- Delay: Expiry point 사이의 간격



22. Schedule Table

```
SCHEDULETABLE SchedTab1 {
```

```
  COUNTER = counter1;
```

```
  DURATION = 10;
```

```
  REPEATING = TRUE;
```

```
  AUTOSTART = TRUE {
```

```
    START_VALUE = 5;
```

```
  };
```

```
  EXPIRE_POINT = ACTION {
```

```
    EXPIRE_VALUE = 0;
```

```
    ACTION = ACTIVATETASK { TASK = TaskH; };
```

```
    ACTION = ACTIVATETASK { TASK = TaskL; };
```

```
  };
```

```
  EXPIRE_POINT = ACTION {
```

```
    EXPIRE_VALUE = 5;
```

```
    ACTION = ACTIVATETASK { TASK = TaskH; };
```

```
    ACTION = SETEVENT { TASK = TaskL; EVENT = Event1; };
```

```
  };
```

```
};
```

Cycle Time

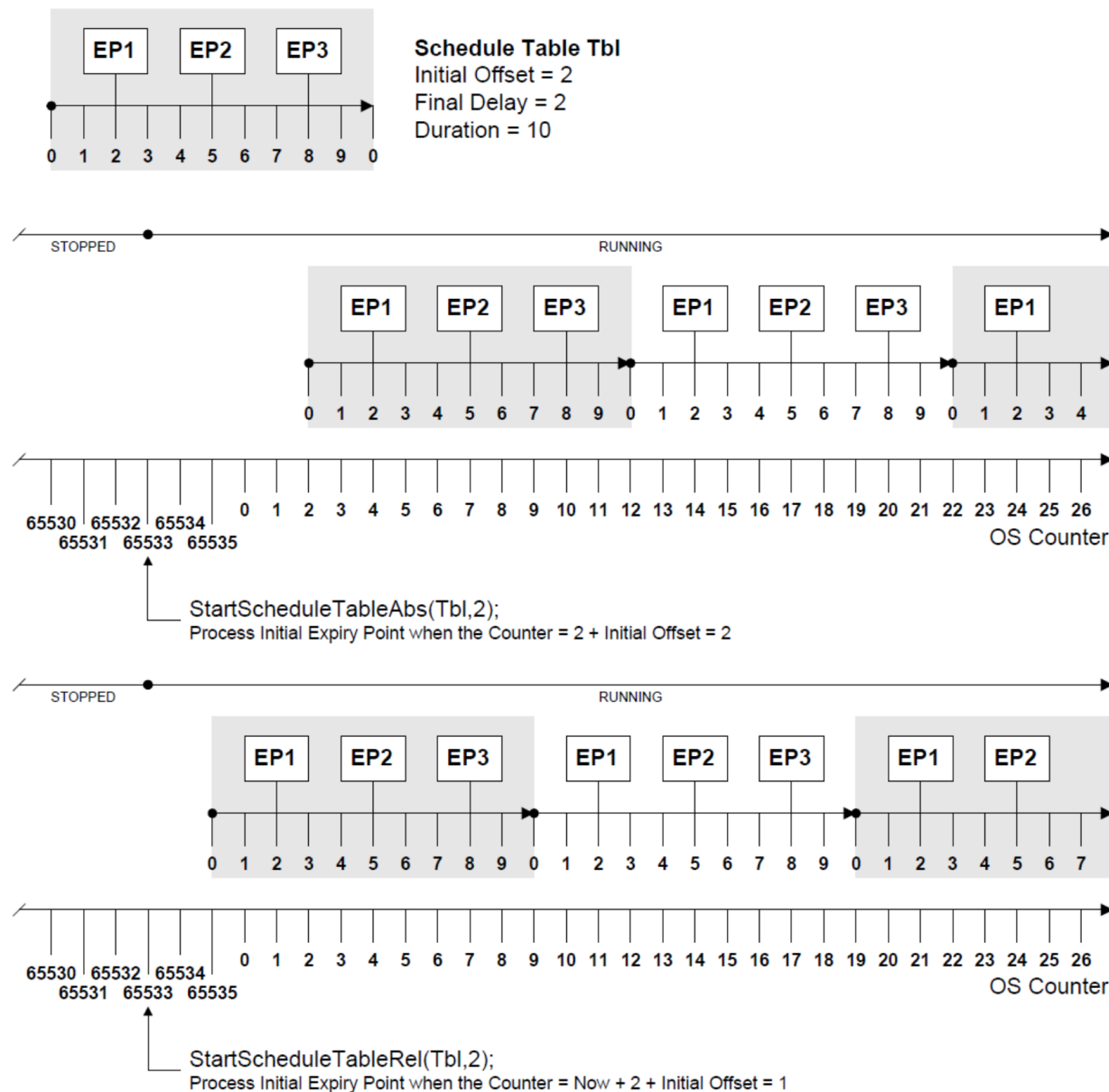
Start Counter
Value

Task Activation

Event Setting

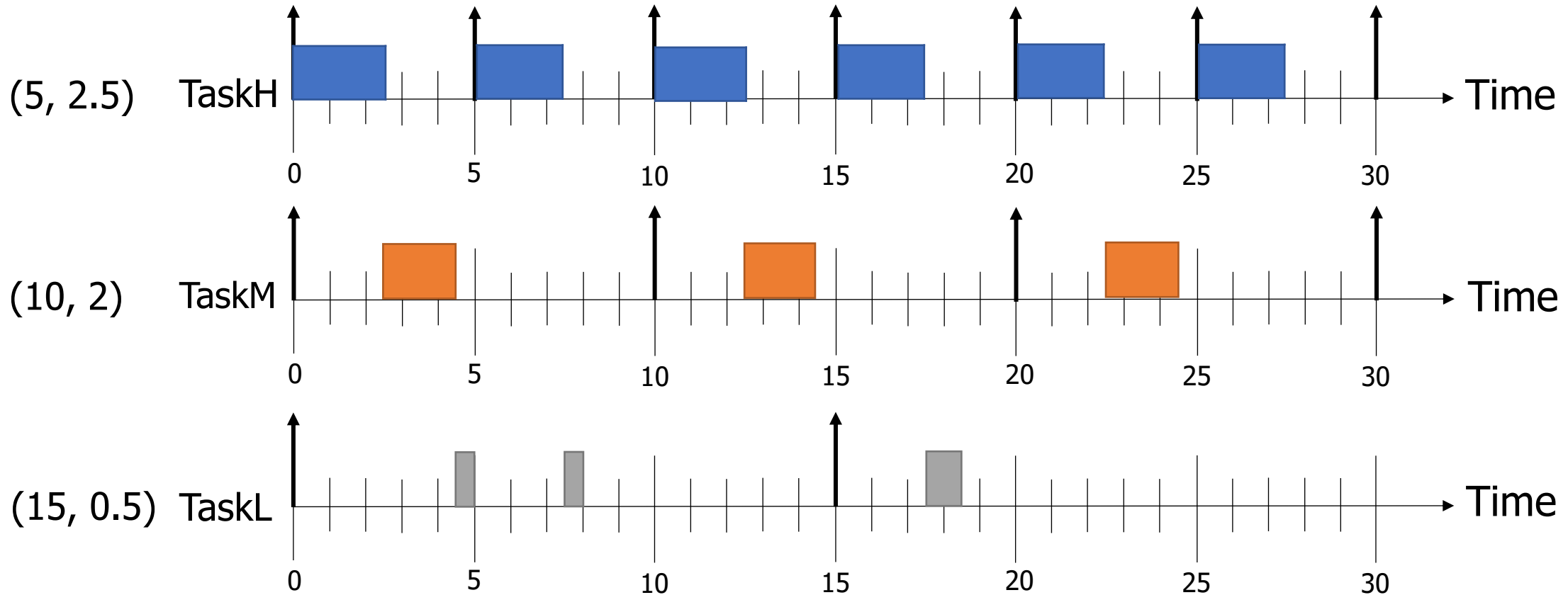
Schedule Table Handling Functions

- StartScheduleTableRel()
- StartScheduleTableAbs()
- StopScheduleTable()



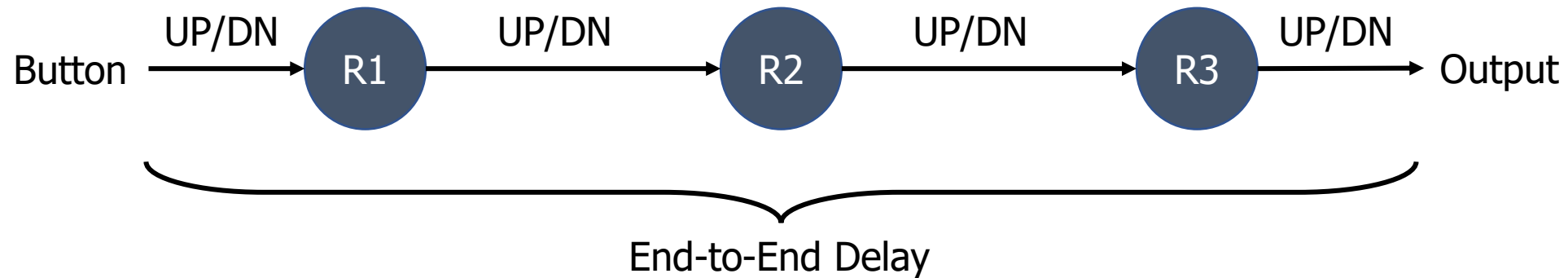
22. Schedule Table

- 아래와 같은 Schedule Table 생성



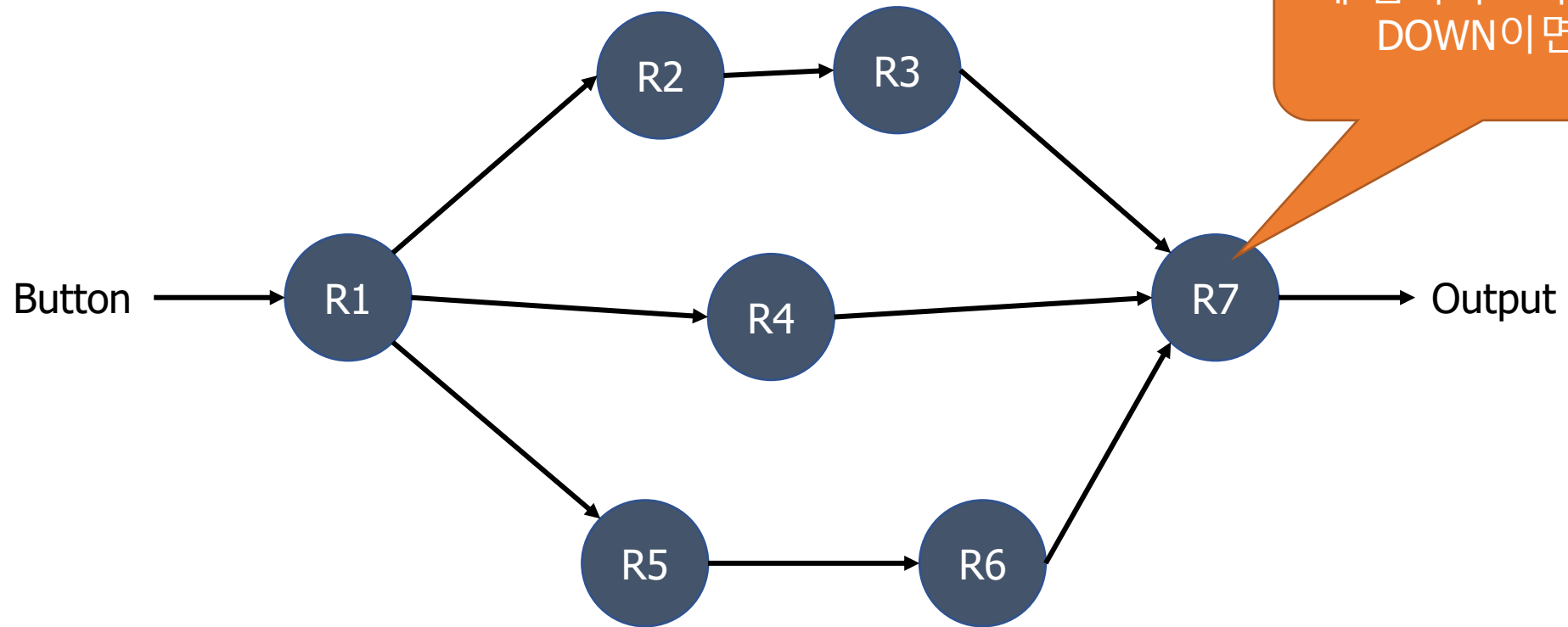
23. End-to-End Delay

- AUTOSAR 기반 DAG (Directed Acyclic Graph) SW 구조
- Runnable to Task 매핑 & 시퀀싱
- Sensor에서 Actuator까지 End-to-End Delay 관찰



23. End-to-End Delay

- 아래 DAG 구조를 정의하고 Delay 측정



Questions

